

TriaC

Progetto per il corso di Ingegneria dei Linguaggi di Programmazione

Andrea Filipuzzi

Anno Accademico 2026/2027

TriaC: Compilatore End-to-End da Sintassi Python-like a LLVM IR

Caratteristiche del Linguaggio:

- Tipizzazione statica forte
- sintassi *layout-sensitive* (indentazione significativa)
- costrutti procedurali
- I/O nativo.

Obiettivo Progettuale:

Separazione netta tra front-end dichiarativo, middle-end a due passate e back-end orientato alla modularità e all'ottimizzazione tramite mem2reg.



Front-End, Grammatica LALR(1) e Layout-Sensitivity

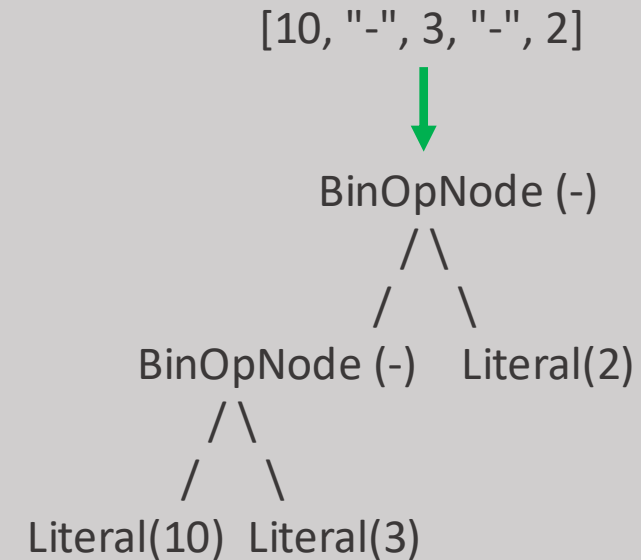
- **Parsing LALR(1) con Lark:**
Precedenze e associatività modellate gerarchicamente (expr → logical_or → ... → factor).
- **Risoluzione del Dangling-Else:**
Nessun conflitto Shift/Reduce; l'allineamento dell'else guida l'emissione del _DEDENT, forzando la riduzione deterministica del blocco associato.
- **Post-Lexer (Indenter):** Stack interno per il tracciamento dei livelli di colonna; emissione dinamica dei token virtuali _INDENT e _DEDENT.
- **Diagnostica d'Errore:**
Riconoscimento immediato tra UnexpectedCharacters (lessicale) e UnexpectedToken (sintattico) con coordinate precise.

AST & Trasformazione

- **Dal CST all'AST:** Eliminazione del rumore sintattico (delimitatori, parole chiave, nodi intermedi di derivazione).

- **Binary Operator Folding:**

- Problema: regole di ripetizione (*) generano liste piatte di token.
- Soluzione: pattern ad accumulatore a passi di due che costruisce ricorsivamente un albero strettamente associativo a sinistra.



- **Nodi Tipizzati:** Modello ad oggetti pulito (`ProgramNode`, `FuncDefNode`, `BinOpNode`, ecc.) che isola il front-end dal resto della pipeline

Middle-End: Analisi Semantica e Scoping

- **Architettura a Due Passate:**

- *Passata 1 (Harvesting)*: Registrazione delle firme delle funzioni nella tabella globale; verifica dell'entry-point main; abilitazione di forward referencing e mutua ricorsione.
- *Passata 2 (Validation)*: Validazione semantica dei corpi, type checking e risoluzione dei simboli.

- **Symbol Table & Scoping:**

- Pila/Albero di ambienti lessicali con puntatore a parent.
- Gestione differenziata tra ridefinizione illecita (stesso scope) e *shadowing* lecito (scope annidati).

- **Control Flow Statico:** Algoritmo ricorsivo per la verifica del ritorno garantito su tutti i rami di esecuzione (check_all_paths_return).

Back-End: Generazione Codice LLVM IR

- **Modello Memory-Based (alloca/load/store):**
 - Ogni variabile e parametro è gestito come puntatore su stack.
 - Nessuna emissione manuale di nodi phi: delega formale ad SSA al passo mem2reg di LLVM.
- **Alloca Hoisting:** Tutte le allocazioni sono ancorate all'inizio del blocco entry per azzerare lo spreco di stack nei cicli.
- **Control-Flow Graph (CFG):**
 - Generazione dei Basic Block per if-then-else e while.
 - Prevenzione di codice morto tramite controllo is_terminated.
- **Runtime C Bridge:**
 - Integrazione con printf e scanf.
 - Indirizzamento stringhe tramite gep con doppio indice [0, 0] per conversione esplicita da $[N \times i8]^*$ a $i8^*$.

Conclusioni & Testing

- **Risultati Raggiunti:** Compilatore completo, modulare, capace di generare eseguibili nativi efficienti e robusto rispetto agli errori.
- **Suite di Test:**
 - Test sintattici e semantici su casi limite (shadowing profondo, cicli annidati, widening implicito).
 - Test negativi con asserzione sui messaggi d'errore (intercettazione errori lessicali, semantici e percorsi senza ritorno).
- **Pipeline di Compilazione Eseguita:**

```
python3 main.py test/casi_validi/test_calcolatrice.tc  
./test/casi_validi/test_calcolatrice
```